

Chess Game with Object-Oriented Programming

A Terminal-Based Two-Player Chess Engine in C++

OOP with C++ — Final Project Report

Abstract

A fully functional chess game was developed in C++ by applying the core principles of Object-Oriented Programming: **encapsulation**, **inheritance**, and **polymorphism**. The program runs entirely in the terminal, rendering a coloured chess board after every move using ANSI escape sequences. The engine correctly enforces all fundamental chess rules, including legal move validation, check detection, checkmate, stalemate, and pawn promotion. Two human players take turns interacting with the game by typing square coordinates at a series of intuitive prompts. The program is structured around three principal abstractions — `Piece`, `Board`, and `Game` — each encapsulating its own data and responsibilities.

1 Introduction

Chess is one of the oldest and most strategically rich board games in existence. Its combination of deterministic rules and near-infinite positional complexity makes it an ideal subject for a software engineering project. Implementing chess in C++ demands careful design decisions: pieces must know how they move, the board must enforce the rules of the game, and the overall flow of play must be managed cleanly. These demands map naturally onto the concepts of Object-Oriented Programming.

The game described in this report is a terminal-based, two-player chess application. It is written entirely in C++ and uses ANSI colour escape sequences to render an attractive, colour-coded board directly in the console. No external libraries or graphical frameworks are required. The code is split across two files: a header file

`Class.h`, which declares all classes and their interfaces, and an implementation file `Class.cpp`, which provides the function bodies. A separate `main.cpp` contains only the entry point and the top-level menu loop.

The sections that follow describe the structure of each class, the logic behind key functions, the way the user interacts with the terminal, and the results that were observed during testing.

1.1 Basic Rules

The game is played on an 8×8 grid whose columns are labelled a–h (files) and whose rows are labelled 1–8 (ranks). Each player controls sixteen pieces: one king, one queen, two rooks, two bishops, two knights, and eight pawns. White always moves first. The object of the game is to deliver checkmate — a position in which the opponent's king is under attack and has no legal move that would remove that attack.

If the player whose turn it is has no legal move but their king is not in check, the result is a stalemate and the game is declared a draw. The movement rules for each piece type are as follows:

- **Pawn:** Advances one square forward; may advance two squares from its starting rank. Captures diagonally one square forward. White pawns travel towards rank 8; black pawns travel towards rank 1.
- **King:** Moves exactly one square in any direction, provided the destination is not attacked by an enemy piece.
- **Queen:** Slides any number of squares along a rank, file, or diagonal, blocked by intervening pieces.
- **Rook:** Slides any number of squares along a rank or file.
- **Bishop:** Slides any number of squares along a diagonal.
- **Knight:** Leaps in an L-shape — two squares along one axis and one square along the perpendicular — and is the only piece that can jump over other pieces.

When a pawn reaches the back rank of the opponent it is promoted: the player selects a replacement piece (queen, rook, bishop, or knight).

2 Code Structure

The codebase is divided into two principal source files and one entry-point file. The header `Class.h` declares all enumerations, structures, and classes. The implementation file `Class.cpp` provides every method body. The file `main.cpp` contains only `main()` and delegates all game logic to the classes defined in `Class.h`.

Three enumerations and one helper structure provide the basic vocabulary shared across the entire codebase:

```
enum class Color      { WHITE, BLACK, NONE };
enum class PieceType { PAWN, ROOK, KNIGHT, BISHOP, QUEEN, KING, EMPTY };

struct Position {
    int row, col;
    bool isValid() const;
    bool operator==(const Position&) const;
};
```

The `Color` enumeration identifies which player owns a piece. `PieceType` identifies the kind of piece. `Position` is a lightweight coordinate pair that validates itself (row and column must lie in $[0, 7]$) and supports equality comparison, which is used extensively when checking whether a candidate move lands on a specific square.

A thin fixed-capacity container called `MoveList` holds up to 64 `Position` values and is returned by every move-generation function. Using a fixed array rather than `std::vector` avoids heap allocation on the critical move-generation path and keeps the code dependency-free.

2.1 main()

The entry point in `main.cpp` is deliberately minimal. After enabling ANSI colour support on Windows via `enableAnsiColors()`, it enters an infinite menu loop. On each iteration it calls `showMainMenu()` to display two options — start a new match or exit — and reads a single character from the user.

```
while (true) {
    showMainMenu();
    char choice; cin >> choice;
    if (choice == '1') {
        collectPlayerNames(whiteName, blackName);
        Game game;
```

```

        game.setPlayers(whiteName, blackName);
        game.run();
    } else if (choice == '2') {
        if (confirmExit()) { /* goodbye screen */ return 0; }
    }
}

```

If the player chooses to start a match, `collectPlayerNames()` gathers the names for both players, constructs a fresh `Game` object, assigns the names, and calls `game.run()`. The `confirmExit()` helper prevents accidental exits by asking for a Y/N confirmation. Any character other than '1' or '2' triggers an invalid-input screen.

2.2 Abstract Class Piece and Its Sub-classes

The abstract base class `Piece` is the parent of all six piece types. It stores three protected attributes:

```

    Color      sideColor_;    // owner of this piece
    PieceType  pieceType_;    // what kind of piece it is
    bool       everMoved_;    // has the piece been moved yet?

```

The `everMoved_` flag is set to `true` by `Board::movePiece()` whenever a piece is relocated. It is used in pawn double-advance logic (only allowed from the starting rank) and can support castling eligibility checks. Public accessors expose the colour, type, and movement status. Two pure virtual functions make the class abstract and force each sub-class to provide its own symbol character and candidate-move list:

```

    virtual MoveList getCandidateMoves(const Position& sq,
                                       const Board& board) const = 0;
    virtual char getSymbol() const = 0;

```

The six concrete sub-classes — Pawn, Rook, Knight, Bishop, Queen, and King — each override both functions. Their move-generation logic is described below.

Pawn

The pawn's movement is the most rule-laden of all pieces. Its advance direction depends on colour: white pawns move toward lower row indices (`adv = -1`), black pawns toward higher ones (`adv = +1`). The function first checks the square directly in front; if empty, that square is added to the candidate list. If the pawn is also on its starting rank (row 6 for white, row 1 for black) and the second square ahead is likewise empty, the double-step is added. Captures are generated by inspecting both forward-diagonal squares: if the diagonal square is valid and contains an enemy piece, it is added as a candidate.

Knight

The knight uses a hardcoded table of eight L-shaped offsets ($\pm 1, \pm 2$) and ($\pm 2, \pm 1$). For each resulting position, the function adds it to the candidate list if it is within the board boundaries

and either empty or occupied by an enemy piece. Because the knight leaps over other pieces, no path-clearing check is needed.

Rook, Bishop, and Queen

All three sliding pieces share a single internal helper, `slidingMoves()`. This function accepts an array of direction vectors and a count, then walks each ray from the piece's square one step at a time. If the next square is empty, it is added and the walk continues. If the square contains an enemy piece, it is added as a capture candidate and the ray is terminated. If it contains a friendly piece, the ray is terminated without adding the square. The rook passes four axis directions, the bishop passes four diagonal directions, and the queen passes all eight.

King

The king iterates over all eight unit offsets ($dr \in \{-1, 0, 1\}, dc \in \{-1, 0, 1\}$, excluding the zero-zero case) and adds any square that is valid and either empty or enemy-occupied. Note that candidate moves are generated without checking whether the destination is attacked; that filtering is applied later by `Board::getLegalMoves()`.

2.3 Class Board

The `Board` class is the heart of the engine. It owns the 8×8 grid of piece pointers

```
Piece* squares_[8][8];
```

and provides every function needed to query the game state, generate legal moves, and detect check, checkmate, and stalemate. It is non-copyable (copy constructor and copy-assignment operator are deleted) to prevent accidental shallow copies of the pointer grid.

Construction and Reset

The default constructor calls `clearBoard()` (sets all 64 pointers to `nullptr`) followed by `setupPieces()`. The `setupPieces()` method uses a local lambda `backRank()` to place the eight back-rank pieces (rook, knight, bishop, queen, king, bishop, knight, rook) for a given colour on a given row, then fills the adjacent pawn rank. `Board::reset()` deletes all existing pieces and calls `setupPieces()` again, making it safe to replay a game without destroying and reconstructing the `Board` object. The destructor iterates over the grid and deletes every non-null pointer, preventing memory leaks.

Square Queries

Three query functions underpin all move generation. `isSquareEmpty()` returns true if a valid position holds a null pointer. `isSquareEnemy()` returns true if the square holds a piece whose colour differs from the supplied `side` argument. `getPieceAt()` is overloaded to accept either a `Position` struct or a raw row/column pair.

Legal Move Generation

The central move-validation function is `getLegalMoves()`. Given a square and the active colour, it first retrieves the piece's candidate moves and then filters them for legality. Each candidate is tested using the templated `tryMove()` function:

```
template<typename Func>
void tryMove(const Position& from, const Position& to, Func fn) {
    Piece* mv = squares_[from.row][from.col];
    Piece* cap = squares_[to.row][to.col];
    squares_[to.row][to.col] = mv;
    squares_[from.row][from.col] = nullptr;
    fn(); // test the position
    squares_[from.row][from.col] = mv; // undo
    squares_[to.row][to.col] = cap;
}
```

Inside the lambda passed to `tryMove()`, `isKingInCheck()` is called for the active colour. If the king would be in check after the move, the candidate is discarded. This approach avoids duplicating the board state: the move is made directly on the grid, tested, and then reversed, all without heap allocation.

Check and Mate Detection

The function `isKingInCheck()` locates the king of the given colour using `findKingSquare()`, then iterates over every enemy piece and asks whether any of its candidate moves can reach the king's square. If at least one enemy piece can, the king is in check. This brute-force scan is called once per candidate move during legal-move generation, making the overall complexity $O(\text{pieces} \times \text{candidate moves} \times \text{enemy pieces})$, which is acceptable for a two-player terminal game.

The function `playerHasLegalMove()` iterates over all friendly pieces and calls `getLegalMoves()` for each. It returns `true` as soon as a single legal move is found, and `false` if none exists. `Game::run()` checks this at the start of every turn: if no legal move exists and the king is in check, the position is checkmate; if no legal move exists and the king is not in check, the position is stalemate.

Board Rendering

The `draw()` function produces the coloured board display. It uses ANSI 256-colour escape codes: light squares are rendered with background colour 229 (a pale yellow), dark squares with background colour 130 (a warm brown). White pieces are printed in bright cyan (colour 51) and black pieces in near-black (colour 16), each in bold. Rank numbers and file letters are printed in bright yellow on either side of the board. A legend mapping letter symbols to piece names is printed below the board so that new players can identify pieces immediately.

2.4 Class Game

The `Game` class manages the turn-by-turn flow of a single match. It owns a `Board` instance, the two player names, the active colour, and a flag that signals when the match is over.

run()

The `run()` method is the game loop. At the top of each iteration it redraws the board, announces the active player, and checks for check. If the active player has no legal moves, the loop immediately exits after printing either a checkmate or a stalemate screen. Otherwise the player selects a piece and a destination through a two-prompt interaction. If at any point the player types `!` at the destination prompt, they are returned to piece selection without losing their turn. Once a valid destination is confirmed, `Board::movePiece()` is called, followed by `handlePromotion()`, and the turn passes to the opponent via `passTurn()`.

Input Parsing

User input follows standard algebraic notation: a file letter (a–h) followed by a rank digit (1–8), for example `e2`. The `parseInput()` function validates this format, converts the file letter to a column index (`col = letter - 'a'`) and the rank digit to a row index (`row = 8 - digit`), and returns the resulting `Position`. The inverse operation, `squareToString()`, converts a `Position` back to algebraic notation and is used when listing legal moves for the player.

Pawn Promotion

After every move, `handlePromotion()` inspects the destination square. If the piece is a pawn that has reached the opponent's back rank (row 0 for white, row 7 for black), the player is presented with a full-screen promotion menu offering Queen, Rook, Bishop, or Knight. The chosen piece is heap-allocated and placed on the board via `Board::placePiece()`, which deletes the pawn and stores the new piece pointer in one step. Input is validated in a loop until a recognised character is received.

3 User Interaction

All interaction takes place through the terminal. A consistent visual language of gold ANSI text on a black background is used for every menu screen, ensuring that menus are immediately distinguishable from the game board itself. Each screen clears the terminal before printing so that there is never any leftover text from a previous state.

At program start, `showMainMenu()` displays the game title and two numbered options. If the player selects option 1, `collectPlayerNames()` prompts for the names of the White and Black players in sequence. A brief piece-legend reminder screen is then shown before the match begins, summarising which letter symbols correspond to which pieces for both colours.

During a match the board is redrawn at the start of each turn. Below the board the active player's name and colour are displayed, and if that player's king is in check a prominent warning line is printed. The player is first asked to enter the coordinate of the piece they wish to move. Invalid inputs (unknown format, empty square, wrong colour, pinned piece) are rejected with an explanatory message and the prompt is repeated. Once a valid piece is selected, its letter symbol and current square are confirmed and its complete list of legal

destination squares is printed. The player then types the destination coordinate. Typing ! at this prompt cancels the selection and returns to piece picking without changing the board state.

When the game ends, a full-screen result screen announces either the winner's name (in the case of checkmate) or a draw (in the case of stalemate) before returning to the main menu. The `confirmExit()` function prevents accidental quits by requiring an explicit Y/N confirmation whenever option 2 is selected from the menu.

4 Results and Discussion

The program was tested by playing complete games from the opening position through to checkmate and stalemate. In all cases the engine correctly accepted only legal moves, detected check promptly, and terminated the game at the appropriate point. Pawn promotion was tested by advancing pawns to the back rank and confirming that all four promotion choices — queen, rook, bishop, and knight — replaced the pawn correctly and that the promoted piece moved according to its type in subsequent turns.

The coloured board display proved clear and readable in a standard terminal. The alternating light and dark squares, combined with colour-differentiated piece symbols, made it straightforward to distinguish white pieces from black pieces and to track the position across many moves.

The principal design decisions that contributed to correctness were:

- **Candidate / legal separation.** Each piece class generates geometrically valid candidate moves without checking game-level constraints. The board then filters these through the try-and-test mechanism in `getLegalMoves()`. This clean separation means that adding a new piece type requires only implementing `getCandidateMoves()` and `getSymbol()`; all check-filtering logic is inherited automatically.
- **In-place move simulation.** The `tryMove()` template modifies the board directly, invokes a test function, and then undoes the modification. This avoids copying the entire board for each candidate and keeps memory usage constant regardless of the number of candidates being evaluated.
- **Polymorphic dispatch.** Storing the board as an array of base-class `Piece` pointers allows uniform handling of all piece types. Virtual dispatch selects the correct `getCandidateMoves()` implementation at runtime, and the rest of the engine does not need to know which specific sub-class it is dealing with.

Several features could be added in future work to extend the game. En passant pawn capture is not yet implemented. Castling, though the `everMoved_` flag was designed with it in mind, was not included in this version. A graphical front-end would substantially improve usability, particularly on systems where ANSI escape codes do not render correctly. Finally, adding a

computer opponent — even a simple minimax player with a shallow search depth — would open the game to single-player use and demonstrate how artificial intelligence can be layered on top of an existing engine.